- This is a draft describes our plan about the CS497 course in the Fall semester

Primary Objective

We study whether Jev can improve the quality–cost trade-off of a fixed diffusion language model by combining **partial-draft pruning** with **final-answer selection**.

The project proceeds in three stages:

1. Establish whether Jev can select better answers from a fixed pool of completed candidates.
2. Test whether Jev can identify promising trajectories before generation finishes.
3. Evaluate whether pruning followed by final selection outperforms complete sampling followed by the same final selector under comparable budgets.

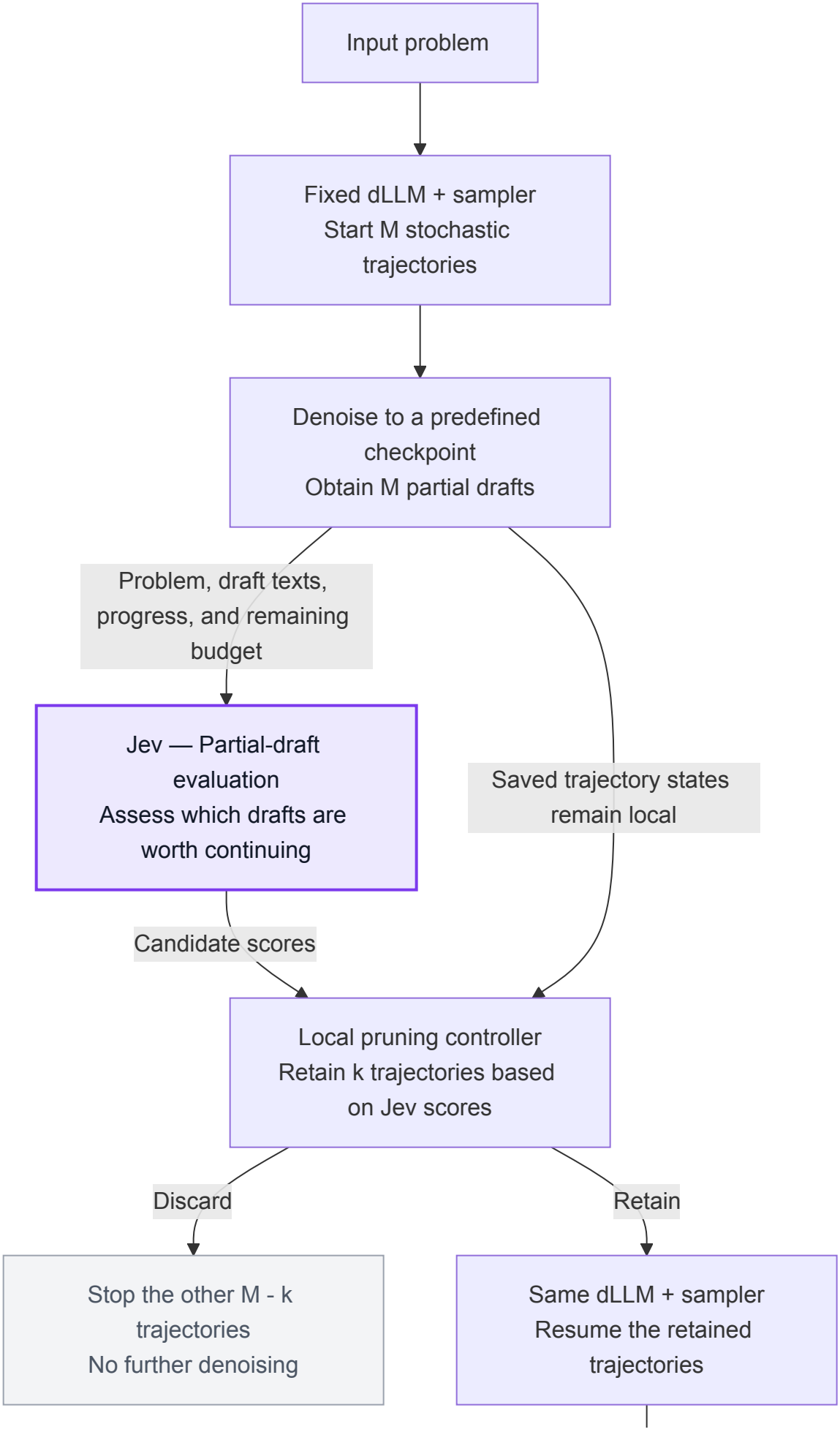
The main system is:

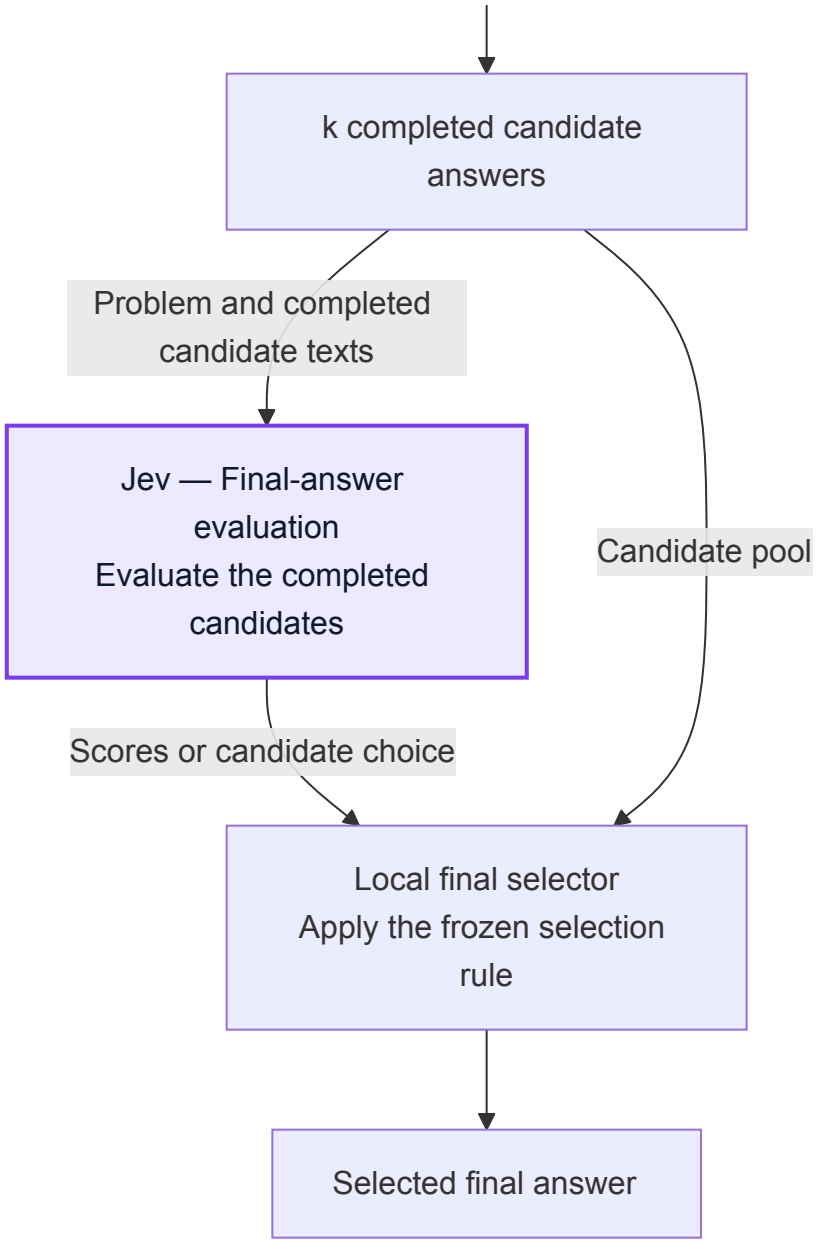
Start multiple dLLM trajectories → evaluate partial drafts → retain promising trajectories → finish generation → select the final answer.

ProcessBench is a diagnostic evaluation, not a prerequisite for success. Token/span-level decoding control, new KV-cache policies, adaptive sample counts, and AR-model comparisons are outside the required scope.

System Overview: Where Jev Fits

Jev acts as an **external judge** at two stages: evaluating partial drafts to guide trajectory pruning, and evaluating completed candidates for final-answer selection. The dLLM and its underlying sampler remain fixed.


Input problem
Fixed dLLM + sampler
Start M stochastic trajectories
Denoise to a predefined checkpoint
Obtain M partial drafts
Problem, draft texts, progress, and remaining budget
Jev — Partial-draft evaluation
Assess which drafts are worth continuing
Saved trajectory states remain local
Candidate scores
Local pruning controller
Retain k trajectories based on Jev scores
Discard
Retain
Stop the other M - k trajectories
No further denoising
Same dLLM + sampler
Resume the retained trajectories



The Two Jev Integration Points

Integration point	What Jev evaluates	What the local controller does	Intended benefit
Partial-draft evaluation	The continuation potential of incomplete candidates, given the problem, generation progress, and remaining budget	Retains selected trajectories and stops the others	Reduce computation spent continuing low-value trajectories
Final-answer evaluation	The quality of completed candidate answers	Selects one answer using the frozen selection rule	Improve the accuracy of the delivered answer

The two Jev nodes represent **two uses of the same judge**, not two separate models. They may use different questions because evaluating a completed answer and predicting the value of continuing a partial draft are different tasks.

Only text and relevant metadata are sent to Jev. Token/mask states, random-number-generator states, and any KV cache remain local. Jev does not directly generate tokens, change model weights, or control token-level unmasking or cache refresh.

Selection-Only Baseline

For the selection-only baseline, candidates are generated to completion without intermediate Jev evaluation or pruning. The same final Jev selector is then applied.

The main comparison is therefore:

Complete sampling + Jev final selection
versus
Jev-guided pruning + Jev final selection

Candidate counts are chosen to support comparable budget comparisons. All intermediate and final Jev calls are included in end-to-end latency and API-cost accounting.

ProcessBench is a separate diagnostic evaluation of Jev and is **not part of this inference pipeline**.

Evaluation Principles

- Keep the generator, decoding configuration, and evaluation protocol fixed within each comparison.
- Use the same candidate pools for comparisons between final-answer selectors.
- Use the same final selector when comparing different pruning strategies.
- Select prompts, thresholds, and configurations on development data, then freeze them before held-out evaluation.
- Keep reference answers, hidden unit tests, and correctness labels unavailable to all deployed selectors.
- Report generation cost, end-to-end latency, and API cost separately. Do not add milliseconds directly to forward-pass counts.
- Treat improved accuracy and reduced cost as hypotheses to test, not predetermined outcomes.

Week 1 · Protocol, Setup, and Baseline Validation

Goal: Establish a reproducible generation and evaluation setup before collecting large candidate pools.

Models and Backend

- Use **LLaDA-8B-Instruct** as the main generator and **Jev** as the main judge.
- Run a small comparison between the official LLaDA sampler and **Fast-dLLM v1**. Compare single-sample accuracy, candidate diversity, and measured generation time before selecting the main backend.
- Start with generation length 256 and block size 32. Check truncation and completion rates on development data before fixing these settings.
- Use BF16 for the generator while preserving the selected implementation's higher-precision sampling operations.
- Record model versions, code commits, prompts, sampling settings, cache configuration, hardware, and microbatch size. Pin the Jev model version.

Data and Scoring

Dataset	Intended evaluation coverage	Scoring
GSM8K	A fixed 500-problem test subset	Final-answer accuracy
MATH500	The standard 500 problems	Fixed answer extraction and mathematical equivalence checking
HumanEval	The standard 164 problems	Unit-test pass rate
ProcessBench	GSM8K and MATH subsets	Official first-error-localization F1

Create disjoint development and held-out question sets. Prefer separate development material so that the intended benchmark coverage can be preserved. If a benchmark is internally split, report the actual held-out subset rather than presenting its score as a full-benchmark result.

Tasks

- Validate answer extraction, code execution, and scoring using known correct and incorrect examples.
- Verify that stochastic sampling produces distinct candidates at N = 8.

- Measure memory usage, generation time, and Jev request latency on representative inputs.
- Run a limited ProcessBench diagnostic without treating its score as a guarantee of selection or pruning performance.

Outputs: Frozen generation protocol; verified scoring pipeline; pilot accuracy and diversity results; runtime profile; ProcessBench diagnostic results.

Week 2 · Shared Candidate Pools and Selection Interfaces

Goal: Build reusable candidate data and the interfaces required for final-answer selection.

Data: Development questions from the three generation benchmarks or their designated development sources, with up to **N = 16 candidates per question**.

Tasks

- Generate and store completed candidates with question IDs, candidate IDs, random seeds, extracted answers, correctness labels, confidence summaries, and generation costs.
- Keep labels in the evaluation pipeline, separate from the inputs supplied to judges.
- Record lightweight intermediate snapshots at predefined checkpoints for later pruning analysis. Save only information needed for evaluation and continuation; do not assume that additional predictions or logging are free.
- Implement two Jev interfaces:
  - **Pointwise:** score each candidate separately using task-specific questions; batch judgments when supported.
  - **Listwise:** present the candidates together and ask Jev to select the best one.
- Use fixed, reproducible candidate subsets for $N \in \{2, 4, 8, 16\}$. Record candidate ordering and tie-breaking rules.

Correctness and reasoning consistency will be treated as different judgment targets. A consistency score will not automatically be interpreted as the probability that the answer is correct.

Held-out candidate pools may be generated in advance, but their evaluation results must not be used for prompt or configuration selection.

Metrics: Mean single-sample accuracy; empirical oracle@N; candidate diversity; generation cost.

Outputs: Shared candidate pools; initial trajectory logs; working pointwise and listwise interfaces; candidate-pool quality statistics.

Week 3 · Final-Answer Selection Experiments

Goal: Determine whether Jev provides useful selection information beyond simpler or established alternatives.

Data: The fixed development candidate pools collected in Week 2.

Comparisons

Selector	Method	Purpose
Random selection	Select a candidate uniformly	Uninformed reference; its expected accuracy equals the pool's mean single-sample accuracy
Majority voting	Select the most frequent normalized final answer	Low-overhead baseline for tasks with comparable final answers
dLLM confidence	Rank candidates using a fixed confidence aggregation rule	Generator-internal signal
Jev pointwise	Rank candidates by individual judgment scores	Independent candidate evaluation
Jev listwise	Choose among candidates in one comparison	Relative candidate evaluation
Jev-weighted voting	Weight answer votes by Jev scores	Combine frequency and judgment where applicable
Qwen2.5-7B-Instruct judge	Use a fixed judgment prompt and scoring procedure	General-purpose LLM-judge comparison
Qwen2.5-Math-PRM-7B	Aggregate process scores for mathematical solutions	Specialized mathematical verifier comparison
Oracle	Select a correct candidate whenever one exists	Diagnostic ceiling for the fixed candidate pool

Do not use majority voting on code by simply comparing code strings. Methods that generate new candidates or rewrite answers are not part of this fixed-pool selector comparison.

Metrics

- Selected-answer accuracy at N ∈ {2, 4, 8, 16}.
- Selection cost and latency.
- Within-question ranking quality for questions containing both correct and incorrect candidates.
- Gap closed: `(selected accuracy − mean single-sample accuracy) / (oracle accuracy − mean single-sample accuracy)`. Report it as undefined when the denominator is zero.
- Calibration metrics only when the predicted event matches the correctness label. Treat consistency scores and listwise choice probabilities primarily as ranking signals.

Outputs: Development accuracy-versus-N curves; selector comparison table; ranking diagnostics; selection-cost measurements.

Week 4 · Selection Analysis and Decision Point A

Goal: Choose and freeze the final-answer selection procedure for the pruning study.

Tasks

- Analyze selection failures involving arithmetic, reasoning, formatting, code logic, and candidate ordering.
- Separate candidate-pool failures from selection failures: was a correct answer available, and did the selector choose it?
- Select the final Jev interface, prompt, aggregation method, and tie-breaking rule using development results.
- Predefine the accuracy-loss tolerance used for later cost-saving claims. A statistically nonsignificant difference will not, by itself, establish equal accuracy.
- Finalize the comparison baselines and prepare the pause-and-resume implementation.

Decision Point A

Does Jev provide useful final-selection value relative to its cost?

- If useful, carry the selected Jev procedure into the pruning experiments.
- If results are mixed, retain the strongest development configuration and document its limitations.
- If no useful signal is found, perform only a limited partial-draft diagnostic before committing to expensive online pruning experiments.

Good performance on completed answers will not be assumed to imply good performance on incomplete drafts.

Outputs: Frozen final selector; selection error analysis; predefined evaluation criteria; **Progress Report 1**.

Week 5 · Predictiveness of Partial Drafts and Decision Point B

Goal: Determine whether intermediate judgments can support pruning before substantial computation is wasted.

Data: Logged development trajectories, initially using **M = 8 trajectories per question**.

Tasks

- Define checkpoints using information available during generation. For a 256-token output with block size 32, candidate checkpoints are after blocks 2, 4, and 6.
- Describe these as block-completion progress, not as known fractions of the trajectory's eventual compute cost.
- Compare two draft presentations:
  - Raw partial text with explicit mask placeholders.
  - A fully predicted draft, where available. Count any additional computation required to construct it.
- Ask Jev to assess the likelihood that continuing the draft will produce a correct answer, given the original problem, current progress, and remaining budget.
- Compare Jev against random and dLLM-confidence rankings using the recorded final outcomes.

Final correctness from a recorded continuation is an observed outcome under the fixed sampler, not a guarantee about every possible continuation.

Primary Metrics

- **Within-question ranking quality:** whether successful continuations receive higher scores than unsuccessful ones.
- **Correct-candidate survival:** the probability that at least one ultimately correct trajectory remains after pruning, conditional on the initial pool containing one.
- Remaining generation cost after each checkpoint.
- Comparison with random and confidence pruning at the same retention rate.

Pooled AUROC is supplementary; it is not sufficient evidence of useful candidate ranking.

Decision Point B

Proceed to online Jev pruning only if development results indicate useful ranking information and enough remaining computation to plausibly offset judging costs. Otherwise, test a limited fallback or document why pruning is not justified.

Outputs: Predictiveness-versus-progress curves; survival-rate comparisons; selected draft presentation and initial pruning checkpoint.

Week 6 · Minimal Pruning System and Correctness Checks

Goal: Implement the complete pipeline with one pruning checkpoint before expanding the experimental grid.

Initial System

Start M trajectories → run to a fixed checkpoint → retain k trajectories → finish generation → apply the frozen final selector.

Begin with a small configuration such as M = 8 and k = 2, using the checkpoint selected in Week 5.

Tasks

- Implement Jev, random, and confidence-based pruning with identical continuation rules.
- Verify pause-and-resume behavior using a no-pruning control.
- Maintain independent random streams for each trajectory so that removing one trajectory does not inadvertently change the randomness assigned to another.
- Preserve or explicitly reconstruct required decoding state. Include cache rebuilding and other restoration costs when they occur.
- Define budget handling, incomplete-candidate handling, and judge-failure behavior before the main comparisons.

Pruning should change which trajectories continue, not silently alter the retained trajectories' sampler.

Cost Logging

Record trajectory-level forward evaluations, dLLM GPU time, draft-preparation overhead, Jev request time, end-to-end latency, and API charges separately.

Outputs: Validated pruning pipeline; pause-and-resume tests; cost logger; preliminary development results.

Week 7 · Main Matched-Budget Experiments

Goal: Test whether intermediate pruning improves the use of inference resources beyond final-answer selection alone.

Core Comparisons

System	Intermediate decision	Final selector
Complete sampling + non-Jev selector	No pruning	Strong task-appropriate baseline from Phase 1
Complete sampling + Jev	No pruning	Frozen Jev selector
Random pruning + Jev	Retain candidates randomly	Same frozen Jev selector
Confidence pruning + Jev	Retain by dLLM confidence	Same frozen Jev selector
Jev pruning + Jev	Retain by Jev judgment	Same frozen Jev selector

The main comparison is **complete sampling + Jev versus Jev pruning + Jev**.

Tasks

- Compare operating points with comparable measured generation budgets. Do not force the same number of initial candidates across methods.
- Allow the complete-sampling baseline to use its budget on additional fully generated candidates.
- Use a limited development grid covering checkpoint, initial candidate count, and retained count.
- Keep hardware, precision, maximum output length, and documented batching policies consistent.
- Report oracle results only as offline diagnostics; they are not deployable methods.

Metrics

Selected-answer accuracy versus measured generation cost; correct-candidate survival; surviving-pool oracle accuracy; candidate diversity; total trajectory-NFE as a supplementary measure.

Outputs: Development accuracy–cost curves; core baseline table; evidence separating selection gains from pruning gains.

Week 8 · End-to-End Efficiency and Targeted Ablations

Goal: Determine whether generation savings survive the cost of preparing drafts, calling Jev, and managing trajectories.

Tasks

- Run the selected configurations online rather than relying only on offline replay.
- Measure end-to-end latency, including all intermediate and final Jev calls, orchestration, and required state reconstruction.
- Report API cost separately from local GPU usage; remote judging is not treated as free computation.
- Measure median and tail latency under a fixed concurrency policy.
- Analyze when pruning overhead is recovered by reduced continuation cost.

Targeted Ablations

- Pruning checkpoint and retained fraction.
- Raw versus fully predicted drafts.
- Jev versus confidence pruning with the same final selector.
- Candidate ordering and the selected judgment formulation.

A delayed-pruning rule may be explored only if the single-checkpoint system is already stable and development results justify it. It is not required for project completion.

Success Criteria

The combined system should demonstrate either higher accuracy at comparable measured cost or lower cost while satisfying the predefined accuracy-loss tolerance.

Outputs: End-to-end accuracy–latency curves; API-cost table; overhead breakdown; ablation results; **Progress Report 2**.

Freeze all primary prompts, configurations, and evaluation decisions at the end of this week.

Week 9 · Held-Out Evaluation and Robustness

Goal: Test the frozen system without further tuning on the evaluation results.

Tasks

- Evaluate the selected systems on the predefined held-out GSM8K, MATH500, and HumanEval sets.
- Repeat stochastic experiments with paired seed schedules where feasible.
- Report paired confidence intervals using the question as the evaluation unit. Multiple candidates from the same question are not independent test questions.
- Check whether improvements persist across tasks and whether they come from retaining better candidates, choosing better among survivors, or both.
- Examine failures without changing the frozen primary configuration.

If resources remain, repeat one core comparison on **Dream-v0-Instruct-7B** as a cross-dLLM robustness check. This is an extension, not a prerequisite for completing the LLaDA study.

Outputs: Final held-out results; uncertainty estimates; reproducibility checks; failure analysis; optional cross-dLLM result.

Week 10 · Final Report and Presentation

Goal: Present a clear answer to the project's quality–cost question.

Report Structure

- Research objective and scope.
- Fixed models, datasets, generation protocol, and judging interfaces.
- Final-answer selection results.
- Partial-draft predictiveness and pruning results.
- Matched-budget and end-to-end comparisons.
- Ablations, failure modes, and limitations.
- Reproducibility details and conclusions.

Distinguish improvements in the inference system from changes to the underlying model's capabilities. Report negative or mixed findings directly, including cases where Jev improves selection but cannot justify its pruning overhead.

Outputs: Final report, presentation, frozen configurations, evaluation scripts, and result tables.

Optional Extensions

Only after completing the primary evaluation, consider adaptive sample counts, multiple pruning checkpoints, additional dLLM backends, AR-model comparisons, or token/span-level decoding control. None of these extensions should displace the matched-budget evaluation of partial-draft pruning combined with final-answer selection.

Appendix

Benchmark

Benchmark	Model / system	Evaluation setting	Reported score (%)	Source
GSM8K	LLaDA-8B-Instruct, without Fast-dLLM acceleration	5-shot; generation length 256	**79.3**	Fast-dLLM, Table 1
GSM8K	LLaDA-8B-Instruct with Fast-dLLM v1	Cache + confidence-aware parallel decoding; 5-shot; generation length 256	**78.5**	Fast-dLLM, Table 1
MATH500	LLaDA-8B-Instruct baseline evaluated in d1	Zero-shot; greedy decoding; generation length 256	**32.4**	d1, Table 1
MATH500	LLaDA-8B-Instruct with Fast-dLLM v1	Our fixed MATH500 protocol	**To be measured**	The cited Fast-dLLM table reports MATH, not explicitly MATH500

Benchmark	Model / system	Evaluation setting	Reported score (%)	Source
HumanEval	LLaDA-8B-Instruct, without Fast-dLLM acceleration	Zero-shot; generation length 256	**41.5**	Fast-dLLM, Table 1
HumanEval	LLaDA-8B-Instruct with Fast-dLLM v1	Cache + confidence-aware parallel decoding; zero-shot; generation length 256	**43.3**	Fast-dLLM, Table 1
ProcessBench	Jev as a critic	First-error localization or a no-error decision; GSM8K and MATH subsets	**To be measured: official F1**	ProcessBench, Sections 3–4

Protocol notes. Fast-dLLM v1 is a training-free inference method, not a second model. Its default configuration uses PrefixCache, block size 32, and confidence threshold 0.9. The Fast-dLLM paper reports MATH rather than explicitly MATH500; its MATH scores must not be relabeled as MATH500. The d1 result above refers to the unmodified LLaDA-8B-Instruct baseline. Cross-paper scores are references only; all Jev comparisons will use one fixed local evaluation protocol.

SOTA

Model	Scale / decoding mode	GSM8K (%)	MATH500 (%)	HumanEval (%)	Primary source
LLaDA2.0-flash	100B-class MoE; diffusion decoding	**96.06**	—	**94.51**	LLaDA2.0, Table 2
FLARE-9B	9B hybrid checkpoint; **Diffusion-Trust** decoding	**93.10**	**93.60**	**82.32***	FLARE, Table 1
SDAR-8B-Chat	8B; block-diffusion decoding	**91.3**	**78.6**	**78.7**	SDAR, Table 3

“—” means that the exact benchmark score is not reported in the cited table. The asterisk is FLARE’s own marker for a potentially under-reported result.